MySQL Query Tuning Cheatsheet

When a WordPress or WooCommerce site slows down, a handful of queries is often doing far more work than it should. These are the checks and fixes I come back to most, in the order I use them. Everything here runs on MySQL 8 and MariaDB 10.6 or later. Take a backup first, and try each change on a staging copy.

01 Find the slow queries

Start with evidence. On WordPress, the Query Monitor plugin lists every query on a page with its time and the plugin or theme that ran it, which is often all you need. For the whole site, turn on MySQL's slow query log:

```
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 0.5;               -- seconds; the default is 10
SET GLOBAL log_queries_not_using_indexes = 'ON';
SHOW VARIABLES LIKE 'slow_query_log_file';     -- where the log is written
```

Leave it on for a day of normal traffic, then summarise the log by total time with `mysqldumpslow -s t -t 10 <log file>`, or with `pt-query-digest` from Percona Toolkit. Turn `log_queries_not_using_indexes` off again afterwards: on a busy site it fills the log quickly. On managed hosting without these permissions, ask your host for the slow log.

02 Read EXPLAIN

Put `EXPLAIN` in front of a SELECT to see how MySQL will run it, without running it. On MySQL 8.3 or later, if you get a tree instead of a table, use `EXPLAIN FORMAT=TRADITIONAL`. The columns that matter:

`type`	How rows are found, from worst to best: `ALL` (every row in the table), `index`, `range`, `ref`, `eq_ref`, `const`. `ALL` on a large table is the first thing to fix.
`key`	The index MySQL chose. `NULL` means none.
`rows × filtered`	Roughly how many rows it expects to read, and the percentage it expects to keep. Many rows read and few kept means a missing or unsuitable index.
`Extra`	`Using filesort` and `Using temporary` mean extra sorting or a temporary table. `Using index` is good: the index alone answered the query.

`EXPLAIN ANALYZE` (MySQL 8.0.18 or later) actually runs the query and shows real row counts and times; on MariaDB, use `ANALYZE SELECT …`. Run these on staging: they execute the query.

03 Add the right index

- ☐ **Put equality columns first, range columns last.**
  For `WHERE post_type = ? AND post_status = ? AND post_date > ?`, an index on (post_type, post_status, post_date) serves all three.

- [ ] **An index works from the left.**
  (a, b, c) helps queries on a, on a and b, and on all three; not queries on b or c alone.
- [ ] **Long text columns need a prefix length.**
  WordPress indexes `meta_key(191)` because utf8mb4 limits index size on older MySQL versions.
- [ ] **Don't index everything.**
  Every index slows down writes and takes space. Add one for a query you've measured, then measure again.

Check what's there with `SHOW INDEX FROM wp_postmeta;`. InnoDB can add most indexes without blocking the site:

```
ALTER TABLE wp_postmeta
ADD INDEX meta_key_value (meta_key(191), meta_value(32)),
ALGORITHM=INPLACE, LOCK=NONE;
```

After large imports or deletes, `ANALYZE TABLE wp_postmeta;` refreshes the statistics MySQL uses to choose an index.

04 wp_postmeta

Custom fields, WooCommerce product data and many plugins all live in this one table, and WordPress only indexes `post_id` and `meta_key`. Any query that filters on `meta_value` reads every row with that key.

Measured on the test store (100,000 products)

```
SELECT post_id FROM wp_postmeta WHERE meta_key = 'color' AND meta_value = 'blue';

-- Core indexes only:      reads 100,000 rows, about 1.25 s, keeps 8,307
-- With meta_key_value:    reads   8,307 rows, about 0.15 s
```

The composite index from section 3 is the fix. Plugins such as Index WP MySQL For Speed add this kind of index for you. One limit: a numeric comparison such as `CAST(meta_value AS UNSIGNED) BETWEEN 1000 AND 2000` can't use it, because the index stores text. For WooCommerce prices, rely on the product lookup tables (section 6) rather than postmeta.

Clear out what nothing uses

```
-- Meta rows whose post no longer exists
SELECT COUNT(*) FROM wp_postmeta pm
LEFT JOIN wp_posts p ON p.ID = pm.post_id
WHERE p.ID IS NULL;

-- Old revisions
SELECT COUNT(*) FROM wp_posts WHERE post_type = 'revision';
```

Limit revisions from now on with `define( 'WP_POST_REVISIONS', 5 );` in wp-config.php. Delete in batches, off-peak, after a backup.

05 wp_options and autoload

WordPress loads every autoloaded option on every request, before any page logic runs. Old plugins often leave large ones behind. Since WordPress 6.6, autoload is stored as `on`, `off`, `auto`, `auto-on` or `auto-off`; older sites use `yes` and `no`. This covers both:

```
-- Total autoloaded size
SELECT ROUND(SUM(LENGTH(option_value)) / 1024) AS autoload_kb
FROM wp_options WHERE autoload IN ('yes', 'on', 'auto-on', 'auto');

-- The largest ones
SELECT option_name, ROUND(LENGTH(option_value) / 1024) AS kb
FROM wp_options WHERE autoload IN ('yes', 'on', 'auto-on', 'auto')
ORDER BY LENGTH(option_value) DESC LIMIT 20;
```

Site Health warns when the total gets large. For a big option that isn't needed on every page, stop it autoloading with `wp_set_option_autoload( 'option_name', false );` (WordPress 6.4 or later), or delete it if it belongs to a plugin you've removed. Expired transients pile up here too: `wp transient delete --expired` clears them with WP-CLI.

06 WooCommerce hotspots

- ☐ **Orders use High-Performance Order Storage.**
  HPOS keeps orders in their own indexed tables instead of posts and postmeta, and is the default for new stores since WooCommerce 8.2. Older stores can migrate under WooCommerce › Settings › Advanced › Features.
- ☐ **The product lookup tables are up to date.**
  Price, stock and rating filters read `wc_product_meta_lookup`. If filters are slow or wrong after an import, regenerate it under WooCommerce › Status › Tools.
- ☐ **Action Scheduler isn't overgrown.**
  Background jobs are logged in `wp_actionscheduler_actions`. Completed actions are kept for 30 days by default; thousands of failed or pending ones point to a problem worth fixing.

```
SELECT status, COUNT(*) FROM wp_actionscheduler_actions GROUP BY status;
```

07 Query patterns to avoid

The same test database shows what each one costs. Watch for them in Query Monitor and in your own code.

`YEAR(post_date) = 2024`	A function on an indexed column stops the index narrowing the search. Here it examined 96,535 rows; `post_date >= '2024-01-01' AND post_date < '2025-01-01'` examined 36,160.
`LIKE '%term%'`	A leading wildcard reads the whole table (193,070 rows here). `LIKE 'term%'` used the index and read 111. For real search, use a search plugin or service.
`ORDER BY RAND()`	Sorts every matching row in a temporary table to return a handful. Pick random IDs in PHP, or cache the result.
`posts_per_page => -1`	Loads every matching post into memory. Paginate, or fetch IDs only with `'fields' => 'ids'`.
`Large meta_query`	Each clause joins wp_postmeta again. Three or four clauses on a large store add up fast.
`Found rows you don't need`	WP_Query counts all matches for pagination. Pass `'no_found_rows' => true` when you don't paginate.

08 Check the server

```
-- How big each table is
SELECT table_name, ROUND((data_length + index_length) / 1024 / 1024) AS mb
FROM information_schema.tables WHERE table_schema = DATABASE()
ORDER BY (data_length + index_length) DESC LIMIT 10;

-- How often InnoDB had to go to disk
SHOW GLOBAL STATUS LIKE 'Innodb_buffer_pool_read%';
```

`Innodb_buffer_pool_reads` (from disk) should be a small fraction of `Innodb_buffer_pool_read_requests`. If it isn't, and the busy tables are larger than `innodb_buffer_pool_size`, the database needs more memory: on a server you manage, give the buffer pool most of the RAM the database can use. A persistent object cache such as Redis also takes repeated queries off MySQL.

09 Safe habits

- [ ] **Back up before every change.**
  `mysqldump --single-transaction --quick db_name > backup.sql` takes a consistent copy without locking InnoDB tables, or use `wp db export`.
- [ ] **Work on staging first.**
  With a recent copy of the live data, so the row counts are realistic.
- [ ] **Change one thing at a time.**
  EXPLAIN and time the query before and after, and keep the change only if it helps.
- [ ] **Schedule big changes for quiet hours.**
  Adding an index to a multi-gigabyte table can take minutes and slow the server while it runs.
- [ ] **Write down what you added.**
  Custom indexes can be lost when a table is rebuilt by a migration or a plugin.

Still slow?

Send me the slowest query from your log, and we'll look at it together on a free call: pradipta.online/free-consultation.